



Purbanchal University
Faculty of Science & Technology

A LAB REPORT ON
ARTIFICIAL INTELLIGENCE

Submitted to
Department of Computer Application
Gomendra Multiple College

In partial fulfillment of the requirements for the Masters of Computer Application

Submitted by
Achyut Chapagain (Roll No: 01)
Regd No: 004-3-3-01635-2025

Under the Supervision of
Mr. Madan Tiwari

Table of Contents

1. PROLOG Program to Answer Whether Kathmandu is the Capital of Nepal	1
2. LISP Program to Answer Whether 7 is Greater than 5	3
3. PROLOG Program to Represent Family Relations	5
4. LISP Program to Check Whether Two People are Siblings	7
5. PROLOG Program to Generate Prime Numbers up to 50	9
6. LISP Program to Compute the GCD of Two Numbers.....	12
7. PROLOG Program to Simulate the AND Gate.....	14
8. LISP Program to Implement the Truth Table of OR Gate	16
9. PROLOG Program to Perform Depth-First Search on a Simple Graph	18
10. LISP Program to Perform Breadth-First Search on a Tree	20
11. PROLOG Menu-Driven Program to Calculate Area	22
12. LISP Menu-Driven Program for Arithmetic Operations.....	24
13. PROLOG Program to Infer Scholarship Eligibility	26
14. LISP Program to Reason Whether a Person Can Vote	28
15. PROLOG Program to Solve Tower of Hanoi with 3 Disks	30
16. LISP Program to Display Tower of Hanoi Steps for 4 Disks.....	32

1. PROLOG Program to Answer Whether Kathmandu is the Capital of Nepal

Objective

To understand how facts and queries are represented in PROLOG and how a knowledge base can answer simple questions.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Basic knowledge of facts and queries in PROLOG

Problem Statement

Write a PROLOG program that stores the fact that Kathmandu is the capital of Nepal and answers whether the statement is true or false.

Procedure / Algorithm

1. Open SWI-Prolog and create a new file.
2. Write the fact using the predicate capital_of/2.
3. Load the file in SWI-Prolog.
4. Run the query to check whether Kathmandu is the capital of Nepal.

Program Code

```
% Program to check whether Kathmandu is the capital of Nepal  
capital_of(nepal, kathmandu).
```

Sample Query / Input

```
?- capital_of(nepal, kathmandu).
```

Output

```
true.
```

Observation

PROLOG checks the available fact in the knowledge base and returns true because the fact is present.

Conclusion

Thus, the program successfully answers that Kathmandu is the capital of Nepal.

2. LISP Program to Answer Whether 7 is Greater than 5

Objective

To learn the use of conditional statements and comparison operators in LISP.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Basic knowledge of LISP functions and if condition

Problem Statement

Create a LISP program that checks whether 7 is greater than 5.

Procedure / Algorithm

1. Open a LISP interpreter such as CLISP or SBCL.
2. Define a function that compares two numbers.
3. Call the function with 7 and 5.
4. Display the result.

Program Code

```
; Program to check whether 7 is greater than 5
```

```
(defun greater-check (a b)
```

```
  (if (> a b)
```

```
    (format t "~a is greater than ~a" a b)
```

```
    (format t "~a is not greater than ~a" a b)))
```

```
(greater-check 7 5)
```

Sample Query / Input

```
(greater-check 7 5)
```

Output

7 is greater than 5

Observation

The greater-than operator compares both values and the condition becomes true.

Conclusion

Thus, the LISP program successfully checks that 7 is greater than 5.

3. PROLOG Program to Represent Family Relations

Objective

To understand representation of family facts and querying relationships using PROLOG.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of facts, rules, and variables

Problem Statement

Write a PROLOG program to represent family relations and query: Who is the father of Sita?

Procedure / Algorithm

1. Create facts for father relationships.
2. Load the program in SWI-Prolog.
3. Ask a query using a variable to find Sita's father.
4. Observe the answer returned by PROLOG.

Program Code

```
% Family relation program
```

```
father(ram, sita).
```

```
father(hari, gita).
```

```
father(shyam, ram).
```

```
% Rule to find father
```

```
is_father(X, Y) :- father(X, Y).
```

Sample Query / Input

```
?- is_father(X, sita).
```

Output

X = ram.

Observation

PROLOG matches the query with stored facts and finds that Ram is the father of Sita.

Conclusion

Thus, the program successfully represents and queries family relations.

4. LISP Program to Check Whether Two People are Siblings

Objective

To understand how lists and conditional checking can be used in LISP for reasoning.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Basic knowledge of cond, equal, and functions

Problem Statement

Develop a LISP program to check if two people are siblings.

Procedure / Algorithm

1. Define parent information using a function.
2. Create a sibling function that checks whether two people share the same parent.
3. Call the function for sample names.
4. Display whether they are siblings.

Program Code

; Program to check sibling relation

```
(defun parent (person)
```

```
  (cond ((equal person 'sita) 'ram)
```

```
        ((equal person 'gita) 'ram)
```

```
        ((equal person 'hari) 'shyam)
```

```
        (t nil))))
```

```
(defun siblings (p1 p2)
```

```
  (if (and (not (equal p1 p2))
```

```
(equal (parent p1) (parent p2)))  
(format t "~a and ~a are siblings" p1 p2)  
(format t "~a and ~a are not siblings" p1 p2)))
```

```
(siblings 'sita 'gita)
```

Sample Query / Input

```
(siblings 'sita 'gita)
```

Output

SITA and GITA are siblings

Observation

Both Sita and Gita have the same parent in the program, so the condition is true.

Conclusion

Thus, the LISP program successfully checks sibling relationships.

5. PROLOG Program to Generate Prime Numbers up to 50

Objective

To learn numerical reasoning in PROLOG by checking and generating prime numbers.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of arithmetic operators and recursion

Problem Statement

Write a PROLOG program to generate all prime numbers up to 50.

Procedure / Algorithm

1. Define a divisor checking rule.
2. Define a prime rule where a number is greater than 1 and has no divisor other than 1 and itself.
3. Use between/3 to generate numbers from 2 to 50.
4. Print numbers that satisfy the prime condition.

Program Code

```
% Generate prime numbers up to 50
```

```
has_divisor(N, D) :-
```

```
    D * D =< N,
```

```
    N mod D =:= 0.
```

```
has_divisor(N, D) :-
```

```
    D * D =< N,
```

```
    D1 is D + 1,
```

```
    has_divisor(N, D1).
```

```
prime(N) :-
```

$N > 1,$

$\backslash + \text{has_divisor}(N, 2).$

prime_upto_50 :-

between(2, 50, N),

prime(N),

write(N), nl,

fail.

prime_upto_50.

Sample Query / Input

?- prime_upto_50.

Output

2

3

5

7

11

13

17

19

23

29

31

37

41

43

47

true.

Observation

The program checks each number from 2 to 50 and prints only those numbers that are prime.

Conclusion

Thus, the PROLOG program successfully generates prime numbers up to 50.

6. LISP Program to Compute the GCD of Two Numbers

Objective

To implement the Euclidean algorithm in LISP using recursion.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of recursion and mod operation

Problem Statement

Create a LISP program to compute the GCD of two numbers.

Procedure / Algorithm

1. Define a recursive function gcd-calc.
2. If the second number is zero, return the first number.
3. Otherwise call the function with b and a mod b.
4. Display the final GCD.

Program Code

```
; Program to find GCD using recursion
```

```
(defun gcd-calc (a b)
```

```
  (if (= b 0)
```

```
    a
```

```
    (gcd-calc b (mod a b))))
```

```
(format t "GCD is: ~a" (gcd-calc 48 18))
```

Sample Query / Input

```
(gcd-calc 48 18)
```

Output

GCD is: 6

Observation

The Euclidean algorithm repeatedly finds the remainder until the second value becomes zero.

Conclusion

Thus, the LISP program successfully calculates the GCD of two numbers.

7. PROLOG Program to Simulate the AND Gate

Objective

To understand how logical gates can be represented using PROLOG facts.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of truth tables

Problem Statement

Write a PROLOG program to simulate the AND gate.

Procedure / Algorithm

1. Create AND gate facts for all input combinations.
2. Load the program in SWI-Prolog.
3. Query the output for specific inputs.
4. Verify the output with the AND truth table.

Program Code

```
% AND gate simulation  
  
and_gate(0, 0, 0).  
and_gate(0, 1, 0).  
and_gate(1, 0, 0).  
and_gate(1, 1, 1).
```

Sample Query / Input

```
?- and_gate(1, 1, X).
```

Output

```
X = 1.
```


Observation

The AND gate gives output 1 only when both inputs are 1.

Conclusion

Thus, the PROLOG program successfully simulates an AND gate.

8. LISP Program to Implement the Truth Table of OR Gate

Objective

To implement and display the OR gate truth table using LISP.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of conditional and logical operators

Problem Statement

Develop a LISP program to implement the truth table of OR gate.

Procedure / Algorithm

1. Define a function that calculates OR output.
2. Use logical condition to return 1 if any input is 1.
3. Call the function for all input pairs.
4. Display the truth table.

Program Code

```
; OR gate truth table
```

```
(defun or-gate (a b)
```

```
  (if (or (= a 1) (= b 1)) 1 0))
```

```
(format t "A B Output~%")
```

```
(format t "0 0 ~a~%" (or-gate 0 0))
```

```
(format t "0 1 ~a~%" (or-gate 0 1))
```

```
(format t "1 0 ~a~%" (or-gate 1 0))
```

```
(format t "1 1 ~a~%" (or-gate 1 1))
```

Sample Query / Input

Run the program in CLISP/SBCL.

Output

A B Output

0 0 0

0 1 1

1 0 1

1 1 1

Observation

The OR gate gives output 1 when at least one input is 1.

Conclusion

Thus, the LISP program successfully displays the OR gate truth table.

9. PROLOG Program to Perform Depth-First Search on a Simple Graph

Objective

To learn how depth-first search can be implemented in PROLOG using recursion.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of recursion, lists, and graph representation

Problem Statement

Write a PROLOG program to perform depth-first search on a simple graph.

Procedure / Algorithm

1. Represent graph connections using edge facts.
2. Define path search using recursion.
3. Maintain a visited list to avoid loops.
4. Run a query to find a path between two nodes.

Program Code

```
% Depth First Search in PROLOG
```

```
edge(a, b).
```

```
edge(a, c).
```

```
edge(b, d).
```

```
edge(c, e).
```

```
edge(d, f).
```

```
edge(e, f).
```

```
connected(X, Y) :- edge(X, Y).
```

```
connected(X, Y) :- edge(Y, X).
```

dfs(Start, Goal, Path) :-

dfs_visit(Start, Goal, [Start], RevPath),

reverse(RevPath, Path).

dfs_visit(Goal, Goal, Path, Path).

dfs_visit(Current, Goal, Visited, Path) :-

connected(Current, Next),

\+ member(Next, Visited),

dfs_visit(Next, Goal, [Next|Visited], Path).

Sample Query / Input

?- dfs(a, f, Path).

Output

Path = [a, b, d, f].

Observation

DFS explores one branch deeply before backtracking to another branch.

Conclusion

Thus, the PROLOG program successfully performs depth-first search on a graph.

10. LISP Program to Perform Breadth-First Search on a Tree

Objective

To understand breadth-first search traversal using queue-like list processing in LISP.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of lists, assoc, and recursion

Problem Statement

Create a LISP program to perform breadth-first search on a tree.

Procedure / Algorithm

1. Represent the tree using an association list.
2. Start from the root node.
3. Visit nodes level by level using a queue.
4. Print each visited node.

Program Code

```
; Breadth First Search in LISP
```

```
(setq tree '((a b c)
```

```
  (b d e)
```

```
  (c f g)
```

```
  (d)
```

```
  (e)
```

```
  (f)
```

```
  (g)))
```

```
(defun children (node)
```

```
  (cdr (assoc node tree)))
```

```
(defun bfs (queue)
  (when queue
    (let ((node (car queue)))
      (format t "~a " node)
      (bfs (append (cdr queue) (children node))))))
```

```
(bfs '(a))
```

Sample Query / Input

```
(bfs '(a))
```

Output

A B C D E F G

Observation

BFS visits the root first, then all child nodes level by level.

Conclusion

Thus, the LISP program successfully performs breadth-first search on a tree.

11. PROLOG Menu-Driven Program to Calculate Area

Objective

To develop a menu-driven PROLOG program for calculating the area of circle, rectangle, and triangle.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of read, write, and arithmetic operations

Problem Statement

Write a PROLOG program with a menu to calculate area of circle, rectangle, or triangle.

Procedure / Algorithm

1. Display menu options to the user.
2. Read the user choice.
3. Read required input values.
4. Calculate and display the selected area.

Program Code

% Menu-driven area calculation in PROLOG

area_menu :-

```
write('1. Area of Circle'), nl,  
write('2. Area of Rectangle'), nl,  
write('3. Area of Triangle'), nl,  
write('Enter your choice: '), read(Choice),  
calculate_area(Choice).
```

calculate_area(1) :-

```
write('Enter radius: '), read(R),
```



```

    Area is  $3.14 * R * R$ ,
    write('Area of circle = '), write(Area).

calculate_area(2) :-
    write('Enter length: '), read(L),
    write('Enter breadth: '), read(B),
    Area is  $L * B$ ,
    write('Area of rectangle = '), write(Area).

calculate_area(3) :-
    write('Enter base: '), read(Base),
    write('Enter height: '), read(H),
    Area is  $0.5 * Base * H$ ,
    write('Area of triangle = '), write(Area).

calculate_area(_) :-
    write('Invalid choice').

```

Sample Query / Input

?- area_menu.

Choice: 1

Radius: 5

Output

Area of circle = 78.5

Observation

The program performs different calculations based on the selected menu choice.

Conclusion

Thus, the PROLOG program successfully calculates areas using a menu-driven approach.

12. LISP Menu-Driven Program for Arithmetic Operations

Objective

To create a menu-driven LISP program for addition, subtraction, multiplication, and division.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of read, format, let, and cond

Problem Statement

Develop a LISP program with a menu to perform addition, subtraction, multiplication, and division.

Procedure / Algorithm

1. Display arithmetic operation menu.
2. Read the choice and two numbers.
3. Use cond to select the operation.
4. Display the result.

Program Code

; Menu-driven arithmetic operations in LISP

```
(defun calculator ()
```

```
  (format t "1. Addition~%")
```

```
  (format t "2. Subtraction~%")
```

```
  (format t "3. Multiplication~%")
```

```
  (format t "4. Division~%")
```

```
  (format t "Enter choice: ")
```

```
  (let ((choice (read)))
```

```
    (format t "Enter first number: ")
```

```
    (let ((a (read)))
```

```
(format t "Enter second number: ")  
(let ((b (read)))  
  (cond ((= choice 1) (format t "Result = ~a" (+ a b)))  
        ((= choice 2) (format t "Result = ~a" (- a b)))  
        ((= choice 3) (format t "Result = ~a" (* a b)))  
        ((= choice 4) (format t "Result = ~a" (/ a b)))  
        (t (format t "Invalid choice"))))))
```

(calculator)

Sample Query / Input

Choice: 1

First number: 10

Second number: 5

Output

Result = 15

Observation

The program uses menu selection and performs the corresponding arithmetic operation.

Conclusion

Thus, the LISP program successfully performs basic arithmetic operations using a menu.

13. PROLOG Program to Infer Scholarship Eligibility

Objective

To understand inference and reasoning in PROLOG using rules and conditions.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of facts, rules, and comparison operators

Problem Statement

Write a PROLOG program to infer if a student is eligible for scholarship based on marks and income.

Procedure / Algorithm

1. Create facts for student marks and family income.
2. Define a rule for eligibility.
3. Run a query to check whether a student is eligible.
4. Observe the inferred answer.

Program Code

```
% Scholarship eligibility reasoning
```

```
marks(ram, 85).
```

```
income(ram, 25000).
```

```
marks(sita, 70).
```

```
income(sita, 60000).
```

```
marks(gita, 90).
```

```
income(gita, 30000).
```

```
eligible(Student) :-
```

```
marks(Student, Marks),
```

income(Student, Income),

Marks >= 80,

Income <= 40000.

Sample Query / Input

?- eligible(ram).

?- eligible(sita).

?- eligible(X).

Output

true.

false.

X = ram ;

X = gita.

Observation

Students with marks at least 80 and income not more than 40000 are inferred as eligible.

Conclusion

Thus, the PROLOG program successfully performs scholarship eligibility reasoning.

14. LISP Program to Reason Whether a Person Can Vote

Objective

To implement a reasoning program in LISP using age and citizenship conditions.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of and, if, and comparison operators

Problem Statement

Create a LISP program to reason whether a person can vote based on age and citizenship.

Procedure / Algorithm

1. Define a function that accepts age and citizenship status.
2. Check whether age is 18 or above and citizenship is true.
3. Display eligible or not eligible message.
4. Test the function with sample data.

Program Code

```
; Voting eligibility reasoning in LISP
(defun can-vote (age citizen)
  (if (and (>= age 18) citizen)
      (format t "Person is eligible to vote")
      (format t "Person is not eligible to vote")))

(can-vote 20 t)
```

Sample Query / Input

```
(can-vote 20 t)
```

Output

Person is eligible to vote

Observation

The condition becomes true when age is 18 or more and the person is a citizen.

Conclusion

Thus, the LISP program successfully reasons whether a person can vote.

15. PROLOG Program to Solve Tower of Hanoi with 3 Disks

Objective

To solve the Tower of Hanoi problem using recursive rules in PROLOG.

Requirements

1. SWI-Prolog interpreter
2. Text editor
3. Knowledge of recursion

Problem Statement

Write a PROLOG program to solve the Tower of Hanoi problem with 3 disks.

Procedure / Algorithm

1. If there is only one disk, move it directly from source to destination.
2. For N disks, move N-1 disks from source to auxiliary.
3. Move the largest disk from source to destination.
4. Move N-1 disks from auxiliary to destination.

Program Code

```
% Tower of Hanoi for 3 disks
```

```
move(1, Source, Destination, _) :-
```

```
    write('Move disk from '), write(Source),
```

```
    write(' to '), write(Destination), nl.
```

```
move(N, Source, Destination, Auxiliary) :-
```

```
    N > 1,
```

```
    M is N - 1,
```

```
    move(M, Source, Auxiliary, Destination),
```

```
    move(1, Source, Destination, Auxiliary),
```

```
    move(M, Auxiliary, Destination, Source).
```


hanoi :- move(3, left, right, center).

Sample Query / Input

?- hanoi.

Output

Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right
true.

Observation

For 3 disks, the Tower of Hanoi problem requires 7 moves. The recursive program prints all moves.

Conclusion

Thus, the PROLOG program successfully solves Tower of Hanoi with 3 disks.

16. LISP Program to Display Tower of Hanoi Steps for 4 Disks

Objective

To solve and display Tower of Hanoi steps for 4 disks using recursion in LISP.

Requirements

1. CLISP or SBCL interpreter
2. Text editor
3. Knowledge of recursion and progn

Problem Statement

Develop a LISP program to display the steps of Tower of Hanoi for 4 disks.

Procedure / Algorithm

1. Define a recursive hanoi function.
2. If there is one disk, print one move.
3. Otherwise move n-1 disks to auxiliary, move one disk to destination, and move n-1 disks to destination.
4. Call the function for 4 disks.

Program Code

; Tower of Hanoi for 4 disks in LISP

```
(defun hanoi (n source destination auxiliary)
```

```
  (if (= n 1)
```

```
    (format t "Move disk from ~a to ~a~%" source destination)
```

```
    (progn
```

```
      (hanoi (- n 1) source auxiliary destination)
```

```
      (format t "Move disk from ~a to ~a~%" source destination)
```

```
      (hanoi (- n 1) auxiliary destination source))))
```

```
(hanoi 4 'left 'right 'center)
```

Sample Query / Input

(hanoi 4 'left 'right 'center)

Output

Move disk from LEFT to CENTER
Move disk from LEFT to RIGHT
Move disk from CENTER to RIGHT
Move disk from LEFT to CENTER
Move disk from RIGHT to LEFT
Move disk from RIGHT to CENTER
Move disk from LEFT to CENTER
Move disk from LEFT to RIGHT
Move disk from CENTER to RIGHT
Move disk from CENTER to LEFT
Move disk from RIGHT to LEFT
Move disk from CENTER to RIGHT
Move disk from LEFT to CENTER
Move disk from LEFT to RIGHT
Move disk from CENTER to RIGHT

Observation

For 4 disks, the Tower of Hanoi problem requires 15 moves. The program displays each recursive move.

Conclusion

Thus, the LISP program successfully displays the Tower of Hanoi steps for 4 disks.